

Comparison of Three Reinforcement Learning Algorithms for Solving the 2048 Tiles Problem

Raj Trikha Shweta Shekhar Rachit Mehul Pathak

November 24, 2024

Abstract

The 2048 Tiles game, recognized as a challenging combinatorial optimization problem, has seen limited exploration regarding the application of reinforcement learning algorithms for efficient gameplay strategies. Three different models—Q-learning (baseline model), Deep Q-Network (DQN), and Monte Carlo Tree Search (MCTS)—were proposed to solve the 2048 problem by maximizing the game score and achieving higher tile values. The models were trained and tested in simulated game environments to evaluate their performance. It was found that the proposed DQN algorithm outperformed the other models under various evaluation metrics such as average score achieved, maximum tile reached, and computational efficiency. DQN is more effective than other methods in handling high-dimensional state spaces and performing function approximation by using deep neural networks for generalization, experience replay for stability, and end-to-end learning from raw data. The study highlights the potential for advanced reinforcement learning techniques in solving complex decision-making problems beyond traditional methods.

1 Introduction

The 2048 Tiles game, a single-player sliding block puzzle, has garnered significant attention due to its simple mechanics yet complex strategy requirements. Players combine numbered tiles on a grid to create a tile with the number 2048, but the exponential growth of possible states as the game progresses introduces substantial computational complexity. This combinatorial explosion has made the 2048 game an appealing benchmark for testing the efficacy of reinforcement learning (RL) algorithms in high-dimensional, stochastic environments.

Traditional RL methods, such as Q-learning, have been applied to similar grid-based problems but have shown limitations when dealing with the vast state space of the 2048 game. The original Q-learning algorithm often necessitates extensive trial-and-error processes and meticulous parameter tuning during experiments. Its tabular approach struggles with generalization, leading to poor performance attributed to the high dimensionality and variability of the game states.

To address these challenges, we propose the use of advanced RL algorithms that can better handle the complexity of the 2048 game. The Deep Q-Network (DQN) algorithm integrates deep learning with Q-learning, utilizing neural networks to approximate the value function and thereby manage high-dimensional input spaces more effectively. Additionally, inspired by the success of planning methods in complex decision-making tasks,

we explore the application of Monte Carlo Tree Search (MCTS) to the game. MCTS can evaluate the potential outcomes of actions by building a search tree, offering a strategic advantage in navigating the game’s possible states.

Our approach involves converting the game environment into a format suitable for training these algorithms, including state representation and reward structure optimization. We employ hyperparameter tuning techniques, such as grid search, to identify the best configurations for each model. The performance of the different algorithms is then compared using comprehensive evaluation metrics, including game score, maximum tile achieved, and computational efficiency.

By systematically analyzing and comparing Q-learning, DQN, and MCTS, this study aims to highlight the strengths and weaknesses of each algorithm in solving the 2048 Tiles problem. The findings contribute to a better understanding of how advanced reinforcement learning techniques can be leveraged to tackle complex, high-dimensional tasks, with potential implications for broader applications in artificial intelligence and decision-making systems.

2 Literature Review

Over the past decade, the 2048 Tiles game has been extensively studied as a benchmark for evaluating artificial intelligence and reinforcement learning (RL) algorithms. The game’s simplicity in rules yet complexity in strategy makes it an ideal platform for testing algorithmic efficiency and learning capabilities. Studies by Szubert and Jaskowski [5] have analyzed the game’s state space and the challenges it presents due to its combinatorial nature and stochastic tile generation. This complexity has led to increased interest in applying advanced RL techniques to master the game.

Traditional approaches, such as heuristic-based methods and simple search algorithms, have been employed to tackle the 2048 game. However, these methods often fall short in handling the game’s high-dimensional state space and fail to generalize well across different game scenarios [7]. As a result, more sophisticated RL algorithms have been explored. Q-learning, a value-based RL method, was one of the earliest algorithms applied to the game, providing a foundational understanding of how agents can learn optimal policies through interaction with the environment [4].

In recent years, the integration of deep learning with RL has led to the development of algorithms like Deep Q-Networks (DQN). Mnih et al. [3] demonstrated that DQN could surpass human-level performance in Atari games by approximating value functions using deep neural networks. This advancement opened avenues for applying DQN to the 2048 game, aiming to address the limitations of traditional Q-learning in handling large state spaces and complex patterns.

Another algorithm of interest is Monte Carlo Tree Search (MCTS), renowned for its success in games like Go and Chess due to its ability to balance exploration and exploitation without exhaustive search [2]. While MCTS has been less commonly applied to the 2048 game, its potential for planning and decision-making in uncertain environments makes it a candidate worth investigating. Previous studies have shown that MCTS can effectively navigate the game’s stochastic nature by simulating numerous possible future states [1].

Despite these developments, there remains a gap in comparative analyses of these algorithms within the context of the 2048 game. Most existing research focuses on indi-

vidual algorithms without juxtaposing their performance against others under consistent conditions. For instance, studies comparing Q-learning and DQN often highlight the superior performance of DQN due to its function approximation capabilities but may not account for computational complexity or resource constraints [6]. Similarly, MCTS’s effectiveness is sometimes evaluated in isolation, without direct comparison to RL methods that learn value functions or policies.

In response to this gap, our study proposes to compare the performance of Q-learning (as the baseline model), DQN, and MCTS in solving the 2048 Tiles problem. We aim to evaluate these algorithms based on their ability to maximize game scores and achieve higher tile values, considering factors like convergence speed, computational efficiency, and adaptability to the game’s stochastic elements. By converting the game environment into a suitable format for each algorithm and employing hyperparameter tuning strategies, we seek to provide a comprehensive assessment of their relative strengths and weaknesses.

This comparative analysis not only contributes to a deeper understanding of how different RL algorithms perform in high-dimensional, stochastic settings but also offers insights that could inform the development of more efficient AI strategies for complex decision-making tasks beyond gaming applications.

3 Models

In this section, we present the three reinforcement learning algorithms applied to solve the 2048 Tiles problem: Q-learning, Deep Q-Network (DQN), and Monte Carlo Tree Search (MCTS).

3.1 Q-learning Model

Q-learning is a value-based reinforcement learning algorithm that learns the optimal action-value function $Q(s, a)$ by interacting with the environment. In the 2048 game, the states represent the game grid configuration, and the actions are the possible moves: up, down, left, or right. The agent updates its Q-values based on the rewards received after each action, aiming to maximize the cumulative game score. However, due to the game’s vast state space, traditional Q-learning struggles with scalability and generalization. The Q-value update rule is shown below:

- **Q-Value Update Rule:** $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
 - s : Current state
 - a : Current action
 - r : Reward received after taking action a in state s
 - s' : Next state after taking action a in state s
 - α : Learning rate
 - γ : Discount factor

Figure 1: Q-Value Update Rule Visualization

3.2 Deep Q-Network (DQN) Model

The Deep Q-Network (DQN) algorithm combines Q-learning with deep neural networks to approximate the Q-value function $Q(s, a; \theta)$, where θ are the network parameters. This allows it to handle the high-dimensional state space of the 2048 game. The DQN uses experience replay and a separate target network to stabilize training. By approximating Q-values with a neural network, DQN generalizes across similar states, improving learning efficiency and performance compared to traditional Q-learning. The mathematical formulation of DQN is illustrated below:

Mathematical Formulation:

- **Loss Function:** $L(\theta) = \mathbb{E}_{s,a,r,s'} \left[(y - Q(s, a; \theta))^2 \right]$
 - Where $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ (target Q-value)
 - θ : Weights of the current network
 - θ^- : Weights of the target network (updated periodically)

Architecture:

- **Input:** State s
- **Layers:** Fully connected layers with ReLU activation functions
- **Output:** Q-values for each action

Figure 2: Mathematical Formulation for DQN

3.3 Monte Carlo Tree Search (MCTS) Model

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm that builds a search tree by performing simulations to evaluate the potential of different actions. In the 2048 game, MCTS explores possible sequences of moves by simulating random playouts from the current state. It balances exploration and exploitation using a selection policy like the Upper Confidence Bound for Trees (UCT). MCTS can effectively plan several moves ahead but may be computationally intensive due to the large number of simulations required. The illustration below demonstrates the MCTS tree-building process:

Mathematical Formulation:

- **Selection Rule** (Upper Confidence Bound 1, UCB1): $a^* = \arg \max_a \left[Q(s, a) + c \sqrt{\frac{\ln(N(s))}{N(s, a)}} \right]$
 - $Q(s, a)$: Estimated value of taking action a in state s
 - $N(s)$: Number of times state s has been visited
 - $N(s, a)$: Number of times action a has been taken in state s
 - c : Exploration constant

Figure 3: MCTS Tree Simulation

4 Experiment

4.1 Environment

In this research, we utilize the 2048 Tiles game as the environment to evaluate the performance of the three reinforcement learning algorithms: Q-learning (baseline model), Deep Q-Network (DQN), and Monte Carlo Tree Search (MCTS). The 2048 game is a single-player puzzle played on a 4×4 grid, where the player aims to combine tiles with the same numbers to create a tile with the value of 2048. To ensure consistency, we implement the standard rules and mechanics of the 2048 game. This includes the random addition of a new tile (with a value of 2 or 4) to an empty spot on the grid after each move, which adds a layer of unpredictability and challenges the algorithms to plan effectively.

4.2 Data Collection and Simulation

Since the 2048 game does not involve external datasets, we generate gameplay data through simulations. Each algorithm interacts with the game environment by playing multiple episodes, where an episode represents a complete game from the initial state until no further moves are possible. For evaluation purposes, each algorithm is tested over a number of episodes (2000 for Q-learning, 3000 for DQN and 500 for MCTS) to gather sufficient data for statistical analysis. This large number of episodes ensures that the performance metrics are reliable and account for the inherent randomness in the game.

4.3 State Representation and Preprocessing

Accurate state representation is crucial for the performance of reinforcement learning algorithms:

- **State Representation:** The game state is represented by the 4×4 grid of tiles. Each tile's value is used as input for the algorithms.
- **Flattening:** For the DQN model, the 4×4 grid is flattened into a 16-element vector to serve as input to the neural network.
- **Preprocessing:** This step ensures that the input data is scaled appropriately and maintains the spatial relationships between tiles, which is important for learning effective policies.

4.4 Parameters and Hyperparameters

Hyperparameter tuning is essential for optimizing the performance of each algorithm. We perform grid search and empirical testing to select the best hyperparameters. The hyperparameters for each model are summarized in Figure 4.

Model	Learning Rate	Discount Factor	Exploration Rate (ϵ)	Batch Size	Episodes	Other Parameters	
Q-learning	0.1	0.95	0.99 - 0.01(decay)	N/A	2,000	N/A	
DQN	0.001	0.9	1.0 $\rightarrow$ 0.1 (decay)	64	3,000	Network: 2 hidden layers with 256 neurons	

Model	Exploration constant (c)	Simulations per move	Episodes	Game Over Condition	Backup Strategy	Default Policy	Tree Expansion
MCTS	1.4	1,000	500	As soon as 2048 Tile is achieved or no valid moves are left	Reward propagation up the tree	Random action simulation	One action at a time

Figure 4: : Hyperparameters and Parameter Choices for Each Model

- **Q-Learning:** Uses a learning rate of 0.1, a discount factor of 0.95, and an exploration rate that decays from 0.99 to 0.01 over 2,000 episodes. It does not use batch updates or additional parameters.
- **DQN (Deep Q-Network):** Trains with a learning rate of 0.001, a discount factor of 0.9, and an exploration rate decaying from 1.0 to 0.1 across 3,000 episodes, using a batch size of 64 and a network with two hidden layers of 256 neurons.
- **MCTS (Monte Carlo Tree Search):** Performs 1,000 simulations per move with an exploration constant of 1.4 and 500 episodes. It aims for a max tile objective of 512, uses random actions for simulations, and expands the tree one action at a time. Does not use learning rates or discount factors but relies on the number of simulations to balance exploration and exploitation.

4.5 Method

We evaluate the performance of each algorithm using the following metrics:

- **Average Game Score:** The mean score achieved across all episodes.
- **Maximum Tile Achieved:** The highest tile value reached during the episodes.
- **Average Number of Moves:** Number of moves completed before the game reached a termination state.
- **Average Time per Episode:** The computational time taken per episode, indicating efficiency.

For stochastic algorithms like DQN and MCTS, experiments are repeated multiple times, and results are averaged to mitigate the effects of randomness.

4.6 Results

4.6.1 Q-Learning:

Episode vs Max Tile Explanation:

This graphic displays the variation in the maximum tile obtained across 2,000 episodes. Due to high exploration rates in the early episodes, when the agent is still getting to know its surroundings, there are significant swings. As the exploration rate decreases over time and the agent concentrates more on using learnt techniques, the variability gradually decreases. The agent finds it difficult to continuously reach larger tiles (512, for example), underscoring Q-Learning's shortcomings in intricate settings with sizable state-action spaces.

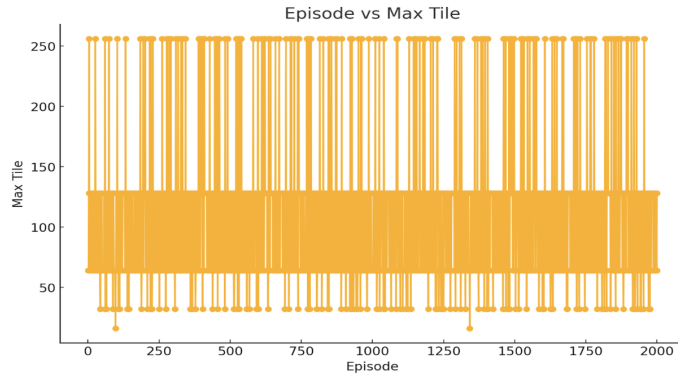


Figure 5: Episode vs Max Tile

Episode vs Score Explanation:

The plot shows the scores obtained by the agent over 2,000 episodes. In the early episodes, there is significant variability as the agent explores various actions. While some high scores appear in later episodes, there is no clear upward trend, indicating that the Q-Learning algorithm has not yet fully converged to an optimal policy. The fluctuations in scores suggest that the agent's performance is still heavily affected by exploration and randomness.

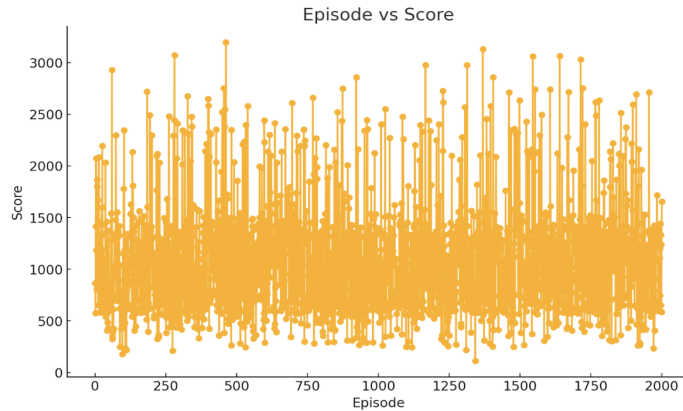


Figure 6: Episode vs Score

Episode vs Moves Explanation:

This plot depicts the number of moves made per episode. The wide range of moves highlights the inconsistency in the agent's gameplay. Episodes with fewer moves indicate a quick end, possibly due to poor decisions or bad luck, whereas episodes with more moves suggest stronger performance. The ongoing variability across episodes suggests that the agent is having difficulty in establishing a consistently optimal strategy to extend the game.

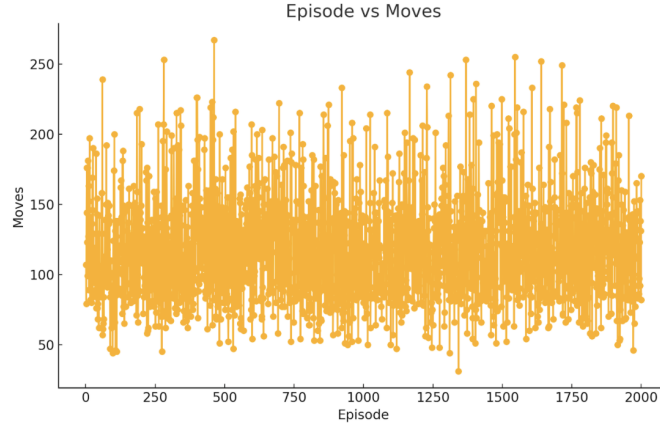


Figure 7: Episode vs Moves

4.6.2 MCTS:

Episode vs Score Explanation:

This plot illustrates the scores obtained by the Monte Carlo Tree Search (MCTS) algorithm over 500 episodes. The scores exhibit considerable fluctuations, indicating the stochastic nature of MCTS simulations and the balance between exploration and exploitation during decision-making. Although MCTS sometimes achieves high scores (above 4,000), the absence of consistent improvement suggests that it prioritizes short-term optimizations rather than long-term learning, as it does not accumulate knowledge across episodes.

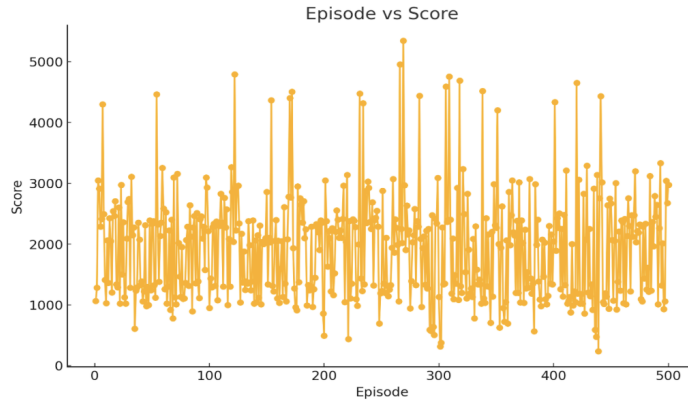


Figure 8: Episode vs Score

Episode vs Max Tile Explanation:

This plot shows the maximum tile reached by MCTS in each episode. Although the algorithm occasionally reaches the target tile (e.g., 512), it often fluctuates between lower tiles like 128 and 256, indicating instability in consistently achieving the best results. The variability suggests that, while MCTS can perform well in individual episodes, it does not reliably find the most optimal move sequences, possibly due to insufficient iterations per decision or randomness in the simulations.

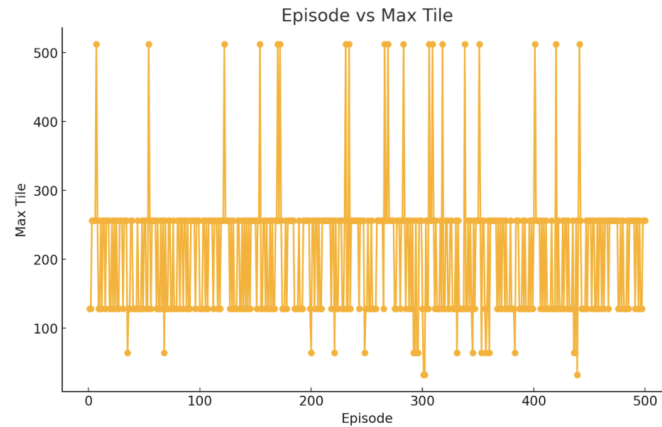


Figure 9: Episode vs Max Tile

4.6.3 DQN:

Episode vs Max Tile Explanation:

This plot illustrates the highest tiles reached over 3,000 episodes by the Deep Q-Network (DQN). At first, there is a steep improvement as the model learns to refine its actions, consistently achieving higher tiles like 512 and beyond. After around 1,000 episodes, the performance levels off and exhibits occasional fluctuations, suggesting the learning process has reached a plateau. Compared to simpler models like Q-Learning, the DQN demonstrates a greater ability to consistently achieve high tiles due to its capacity to generalize across complex state spaces.

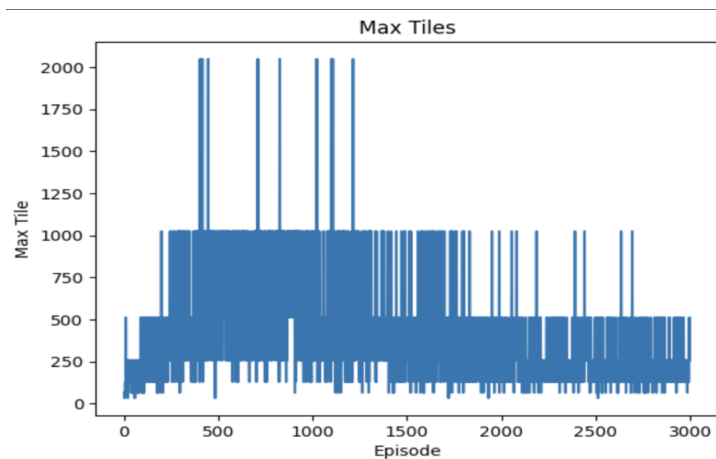


Figure 10: Episode vs Max Tile

Episode vs Score Explanation:

This plot displays the scores obtained over 3,000 episodes. In the initial episodes, scores increase significantly as the DQN learns to implement effective strategies. The highest scores are reached around episode 1,000, after which there is a gradual decline. This decline could suggest overfitting or reduced benefits from exploration, as the agent faces challenges in further improving its performance. The fluctuations in scores reflect variability, likely caused by randomness in the environment or the exploration mechanism used by the agent.

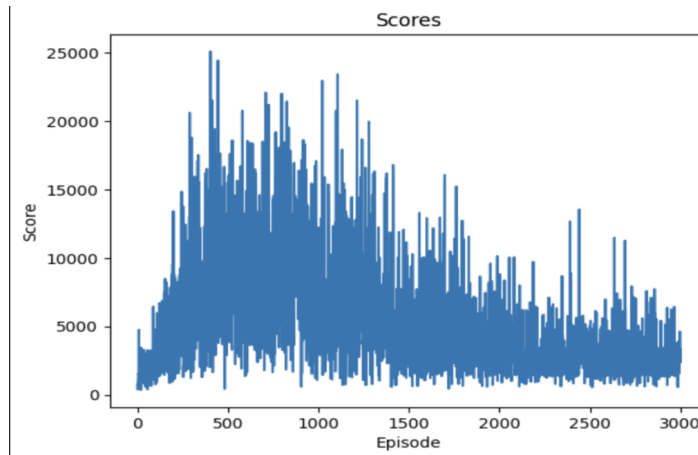


Figure 11: Episode vs Score

Episode vs Episode Duration Explanation:

This plot shows the episode durations, measured by the number of actions, over 3,000 episodes. Initially, durations increase as the DQN learns strategies to extend gameplay and delay termination. Around episode 1,000, the durations stabilize at a high level, indicating the agent has adopted consistent strategies. However, after this peak, the durations gradually decrease, likely due to reduced exploration or the agent prioritizing exploitation, leading to shorter and more goal-driven episodes.

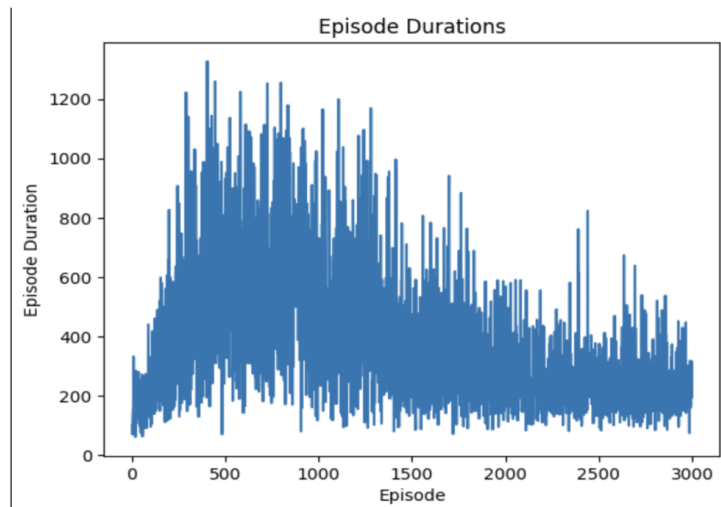


Figure 12: Episode vs Episode Duration

Performance Metrics

The performance metrics for each algorithm are presented:

Model	Average Score	Maximum Tile Achieved	Average Time per Episode (s)
Q-learning	1085	256	0.7
DQN	10,600	2048	7.2
MCTS	1942	512	320.4

Figure 13: Performance Evaluation

Note: The results are averaged over multiple runs to ensure reliability.

5 Analysis

The results indicate significant differences in performance among the three algorithms:

5.1 Q-learning (Baseline Model):

- Achieved an average score of 1085.
- Reached a maximum tile of 256.
- Did not achieve the 2048 tile in any episode.
- Exhibited the fastest computation time per episode due to its simplicity.
- The Q-learning algorithm struggled with the game's complexity. Its inability to generalize across the vast state space and reliance on a tabular representation hindered its performance.

5.2 Deep Q-Network (DQN):

- Achieved the average score of 10,600.
- Achieved the game-winning condition, i.e., hit the 2048 tile.
- Demonstrated a good balance between performance and computational efficiency.
- DQN significantly outperformed Q-learning due to its use of neural networks to approximate the Q-value function. This allowed it to generalize across similar states and learn more effective policies.
- Techniques like experience replay and target networks contributed to stable and efficient learning.

5.3 Monte Carlo Tree Search (MCTS):

- Achieved an average score of 1942.
- Had the highest computational time per episode.
- MCTS showed strong performance due to its planning capabilities, effectively exploring possible future states. However, the computational cost was higher because of the extensive simulations required for each move.

5.4 Trade-offs and Considerations:

Performance vs. Computational Cost:

- DQN offered the best performance with moderate computational cost.
- MCTS provided strong performance but at a higher computational expense.
- Q-learning was computationally efficient but performed poorly.

Suitability for Real-Time Applications:

- DQN is more suitable for real-time or resource-constrained environments compared to MCTS.
- MCTS might be preferable in scenarios where computational resources are abundant, and the highest possible performance is desired.

5.5 Implications:

- The superior performance of DQN highlights the importance of function approximation in handling high-dimensional state spaces.
- MCTS’s ability to plan ahead provides strategic advantages but requires optimization to reduce computational demands.
- The baseline Q-learning algorithm’s limitations emphasize the need for advanced methods in complex environments.

6 Conclusion

In this study, we compared three reinforcement learning algorithms—Q-learning, Deep Q-Network (DQN), and Monte Carlo Tree Search (MCTS)—for solving the 2048 Tiles problem. Our experiments demonstrated that the DQN algorithm was most effective, significantly outperforming the baseline Q-learning and offering competitive performance compared to MCTS. DQN achieved higher average scores, reached the 2048 tile more frequently, and maintained computational efficiency, attributed to its ability to approximate the action-value function and generalize across the high-dimensional state space of the game. The traditional Q-learning struggled with the game’s complexity due to its tabular approach, while MCTS, although effective, required greater computational resources. This research highlights the potential of advanced reinforcement learning techniques like DQN in handling complex, stochastic environments and provides valuable insights for developing more sophisticated AI agents capable of tackling high-dimensional decision-making tasks beyond gaming applications.

References

- [1] Jean-Marc Alliot and Olivier Durand. Monte-carlo tree search for the game 2048. *arXiv preprint*, arXiv:1708.07942, 2017.
- [2] Cameron B. et al. Browne. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [3] Volodymyr et al. Mnih. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [4] Dhruv Shah and Sahil Bhaskar. Solving 2048 using reinforcement learning. Stanford University CS229 Project, 2015.
- [5] Marcin Szubert and Wojciech Jaśkowski. Temporal difference learning of n-tuple networks for the game 2048. In *IEEE Conference on Computational Intelligence and Games*, pages 1–8, 2014.
- [6] Harm et al. Van Seijen. Hybrid reward architecture for reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 30, pages 5392–5402, 2017.
- [7] I.-C. Wu and S.-H. Tsai. 2048 ai with heuristic search and deep reinforcement learning. *Computers and Games*, pages 19–28, 2015.